Mirando dentro de los elementos

Adaptación a C#

Todo algoritmo de ordenamiento de la Parte I ha estado trabajando en el *modelo de comparaciones* — el único acceso del algoritmo a los datos es preguntar, de cualesquiera dos elementos, ¿cuál es menor? El ordenamiento de burbuja no mira los bits de un entero; merge sort no espía los caracteres de una cadena; quickselect no le importa si estás ordenando números o nombres. Los elementos han sido cajas opacas que el algoritmo puede sostener una junto a otra y nada más.

Esa suposición — la opacidad — es lo que encierra al ordenamiento basado en comparaciones en $\Omega(n \log n)$. La cota inferior del árbol de decisión del capítulo 4 era una afirmación sobre el *modelo de comparaciones*: con solo $\log_2(n!)$ hojas alcanzables por nivel del árbol, no puedes llegar a la respuesta correcta en menos comparaciones. Pero no era una afirmación sobre el ordenamiento en sí. Era una afirmación sobre el ordenamiento bajo la restricción de que lo único que puedes hacer es comparar.

Abandona esa restricción y la cota se va.

Dos algoritmos ordenan en tiempo lineal — $O(n)$, venciendo la cota inferior de comparaciones por un factor de $\log n$ — y el precio que pagan es explícito. Ambos solo funcionan cuando puedes mirar dentro de los elementos, tratando a cada uno no como un valor opaco a comparar sino como una cosa estructurada con propiedades intrínsecas. Una magnitud que mapea directamente a una posición en memoria. Una secuencia de dígitos, cada uno en un rango acotado. Una representación de ancho fijo. El argumento de la cota inferior no aplica porque estos algoritmos no están basados en comparaciones; están basados en estructura. Modelo distinto, cota distinta.

El valor es la dirección. El ordenamiento por conteo toma un entero x y lo usa como el índice en un arreglo de conteo — sin comparación, solo una búsqueda directa en memoria. El ordenamiento por base descompone un entero en dígitos y ordena por cada posición de dígito por turno. En ambos casos el algoritmo lee la estructura intrínseca del elemento en lugar de su posición comparativa relativa a otros elementos. Ese único reencuadre — de comparar a mirar dentro — es el movimiento que rompe el techo de $n \log n$.

1. Ordenamiento por conteo: el valor es la dirección

El aprovechamiento más simple. Supón que cada elemento de `items` es un entero no negativo en el rango $[0, k]$. Entonces puedes ordenar el arreglo contando cuántos de cada valor hay y leyéndolos de vuelta en orden.

Hay tres fases. **Histograma:** recorre la entrada una vez y cuenta las ocurrencias de cada valor. **Suma de prefijos:** convierte el histograma en un conteo acumulado, de modo que `counts[i]` te dice cuántos elementos son $\leq i$ — que es exactamente la posición justo después del último i en la salida ordenada. **Colocación:** recorre la entrada en reversa, y para cada elemento, la suma de prefijos decrementada da su índice de destino.

```
/// <summary>
/// Ordena una secuencia de enteros en [0, k] en tiempo O(n + k).
    Estable.
/// </summary>
public static int[] CountingSort(IReadOnlyList<int> items, int k)
{
```

```
    var counts = new int[k + 1];
    foreach (var x in items)
        counts[x]++;

    // Sumas de prefijos: counts[i] pasa a ser el numero de elementos <= i.
    for (int i = 1; i <= k; i++)
        counts[i] += counts[i - 1];

    // Coloca cada elemento, recorriendo la entrada en reversa para
    // preservar la estabilidad.
    var output = new int[items.Count];
    for (int idx = items.Count - 1; idx >= 0; idx--)
    {
        int x = items[idx];
        counts[x]--;
        output[counts[x]] = x;
    }
    return output;
}
```

Listing 1: Ordenamiento por conteo

Todo el algoritmo son tres pasadas, y ni una sola de ellas involucra una sola comparación entre dos elementos de la entrada. La pasada del histograma incrementa `counts[x]` para cada x; la pasada de suma de prefijos convierte `counts` en conteos acumulados; la pasada de colocación lee los conteos acumulados como índices de destino. Cada pasada es $O(n)$ (el histograma y la colocación) u $O(k)$ (la suma de prefijos). Total: $O(n + k)$.

```
var result = CountingSortImpl.CountingSort(
    new[] { 3, 1, 4, 1, 5, 9, 2, 6 }, k: 9);
Console.WriteLine(string.Join(", ", result)); // 1, 1, 2, 3, 4, 5, 6, 9

Console.WriteLine(string.Join(", ", CountingSortImpl.CountingSort(
    new[] { 0, 0, 0 }, k: 0)));                  // 0, 0, 0 (duplicados OK)

Console.WriteLine(string.Join(", ", CountingSortImpl.CountingSort(
    Array.Empty<int>(), k: 10)));                // (vacio)
```

```
1, 1, 2, 3, 4, 5, 6, 9
0, 0, 0
```

El recorrido en reversa. El tercer bucle recorre `items` en reversa, no hacia adelante, y eso no es una elección estilística — es lo que hace *estable* al ordenamiento por conteo. Estabilidad significa que elementos iguales preservan su orden relativo de la entrada a la salida. Con el recorrido en reversa, la última copia del valor v en la entrada se coloca en la ranura más a la derecha reservada para v en la salida, y las copias anteriores llenan las ranuras a la izquierda de esa. Recorrer hacia adelante en su lugar invertiría el orden relativo de los elementos iguales. La estabilidad va a ser esencial para el ordenamiento por base, así que el detalle del recorrido en reversa no es cosmético.

El k en $O(n + k)$. Si k es aproximadamente del tamaño de n — digamos, ordenar enteros en $[0, n]$ — entonces el algoritmo es genuinamente lineal. Si k es enorme — digamos, ordenar enteros de 32 bits donde $k = 2^{32}$ — entonces el ordenamiento por conteo asigna un arreglo de cuatro mil millones de entradas y paga $O(k)$ por la pasada de suma de prefijos, lo que destruye

cualquier beneficio práctico. El ordenamiento por conteo gana decisivamente cuando el rango de valores es pequeño y colapsa a inutilidad cuando no lo es. El siguiente algoritmo arregla eso.

2. Ordenamiento por base: un dígito a la vez

¿Y si tus enteros son grandes pero sus dígitos están acotados? Un entero de 32 bits puede tomar cualquiera de 2^{32} valores, pero cada uno de sus diez dígitos decimales está en $[0, 9]$. Cada uno de sus 32 dígitos binarios está en $[0, 1]$. Cada uno de sus cuatro bytes está en $[0, 255]$. El ordenamiento por conteo es inútil sobre los enteros mismos, pero es perfectamente adecuado para ordenar por un solo dígito.

El ordenamiento por base es la construcción natural: ordena por el dígito de menor orden, luego por el siguiente más bajo, y así hasta el más alto. La variante que mostraré es **LSD** (*least significant digit first*, dígito menos significativo primero), que es la más fácil de analizar. Después de que todas las posiciones de dígitos se procesan, el arreglo está ordenado por el valor completo.

```
/// <summary>
/// Ordena enteros no negativos en tiempo O(d * (n + base)).
/// d = numero de digitos del mayor elemento en la base dada.
/// LSD (digito menos significativo primero).
/// </summary>
public static int[] RadixSort(IReadOnlyList<int> input, int radix = 10)
{
    if (input.Count == 0) return Array.Empty<int>();
    var items = new int[input.Count];
    for (int i = 0; i < input.Count; i++) items[i] = input[i];

    int maxVal = 0;
    foreach (var x in items)
        if (x > maxVal) maxVal = x;

    long digitPlace = 1; // unidades, luego base, luego base^2, ...
    while (maxVal / digitPlace > 0)
    {
        items = CountingSortByDigit(items, digitPlace, radix);
        digitPlace *= radix;
    }
    return items;
}
```

Listing 2: Ordenamiento por base (LSD)

El bucle conductor averigua cuántos dígitos tiene el mayor elemento y ejecuta un ordenamiento por conteo estable por posición de dígito. La pasada por dígito es solo ordenamiento por conteo aplicado a una clave extractora de dígito en lugar del valor mismo:

```
private static int[] CountingSortByDigit(
    int[] items, long digitPlace, int radix)
{
    var counts = new int[radix];
    foreach (var x in items)
        counts[(int)((x / digitPlace) % radix)]++;

    for (int i = 1; i < radix; i++)
        counts[i] += counts[i - 1];

    var output = new int[items.Length];
    for (int idx = items.Length - 1; idx >= 0; idx--)
```

```
    {
        int x = items[idx];
        int d = (int)((x / digitPlace) % radix);
        counts[d]--;
        output[counts[d]] = x;
    }
    return output;
}
```

Listing 3: Una pasada estable: ordenar por `(x / digitPlace) % radix`

La forma es idéntica al ordenamiento por conteo simple. El único cambio es la "clave" que usa el algoritmo: en lugar de x, usa `(x / digitPlace) % radix`, que extrae un dígito específico de x.

```
var result = RadixSortImpl.RadixSort(
    new[] { 170, 45, 75, 90, 802, 24, 2, 66 });
Console.WriteLine(string.Join(", ", result)); // 2, 24, 45, 66, 75, 90,
    170, 802

Console.WriteLine(string.Join(", ", RadixSortImpl.RadixSort(
    new[] { 1234, 56, 7, 89, 3210 })));       // 7, 56, 89, 1234, 3210

Console.WriteLine(string.Join(", ", RadixSortImpl.RadixSort(
    new[] { 12, 11, 22, 21 })));              // 11, 12, 21, 22

// Base distinta, mismo resultado: la base solo afecta el factor
    constante.
Console.WriteLine(string.Join(", ", RadixSortImpl.RadixSort(
    new[] { 170, 45, 75, 90, 802, 24, 2, 66 }, radix: 2)));
```

```
2, 24, 45, 66, 75, 90, 170, 802
7, 56, 89, 1234, 3210
11, 12, 21, 22
2, 24, 45, 66, 75, 90, 170, 802
```

¿Por qué funciona el ordenamiento por base LSD? No es obvio que ordenar por el dígito menos significativo primero deba llevar a algo útil — el dígito más bajo es la posición menos importante, intuitivamente. La respuesta está en la estabilidad.

Después de la primera pasada, el arreglo está ordenado por el dígito de las unidades. Los elementos con dígito de unidades 4 están agrupados juntos, en algún orden. Después de la segunda pasada, el arreglo está ordenado por el dígito de las decenas mientras preserva el orden relativo de la pasada anterior — porque el ordenamiento por conteo es estable. Así que dentro del grupo de elementos con decenas 7, el orden del dígito de las unidades de la primera pasada se preserva. Por inducción, tras j pasadas el arreglo está ordenado por los j dígitos inferiores de cada valor. Tras todas las d pasadas, está ordenado por el valor completo.

La estabilidad es lo que hace que el ordenamiento por base funcione en absoluto. Sin ella, cada pasada revolvería el orden establecido por la anterior, y toda la construcción colapsaría.

¿Cuál es el coste práctico? Cada pasada es $O(n + b)$, donde b es la base. Hay d pasadas, donde d es el número de dígitos del mayor valor (que es $\log_b(\text{max_val})$). Total: $O(d(n + b)) = O((n + b)\log_b(\text{max_val}))$. Para enteros de ancho fijo (digamos, 32 bits), d está acotado por una constante — a lo sumo 32 si ordenas un bit a la vez, a lo sumo 4 si ordenas un byte a la vez. Elige base 256 y obtienes 4 pasadas para enteros de 32 bits, 8 pasadas para 64 bits, cada una corriendo en $O(n + 256) = O(n)$ — genuinamente lineal en la práctica. Esa es la elección que hacen las implementaciones de ordenamiento por base de producción.

3. Las tres preguntas, aplicadas

¿Es correcto?

El ordenamiento por conteo es correcto porque las sumas de prefijos te dicen, para cada valor v, exactamente qué posiciones en la salida deben llenarse con copias de v. Después de la fase 2, `counts[i]` es igual al número de elementos de la entrada con valor $\leq i$ — que es la posición justo después del último i. La colocación con recorrido en reversa entonces llena esas posiciones desde la derecha hacia adentro, un elemento a la vez, y el decremento antes de colocar asegura que la siguiente copia de v caiga una ranura más a la izquierda. Inductivamente, todo elemento de la entrada termina en su posición correcta en la salida ordenada.

El ordenamiento por base es correcto por inducción sobre el número de dígitos procesados. Tras j pasadas (menos significativo primero), el arreglo está ordenado por los últimos j dígitos de los valores — es decir, por su valor módulo b^j. La pasada $(j+1)$-ésima es un ordenamiento estable por el dígito $j+1$, que extiende el orden a los últimos $j+1$ dígitos sin perturbar el orden de los dígitos inferiores de pasadas anteriores. Tras todas las d pasadas, el arreglo está ordenado por el valor completo.

La estabilidad del ordenamiento por dígito es el quid del argumento inductivo: si cualquier pasada revolviera los elementos de dígito igual, el paso inductivo fallaría. La estabilidad no es una propiedad estética del ordenamiento por conteo — es la propiedad que permite que el ordenamiento por base exista.

¿Cuán eficiente es?

El ordenamiento por conteo corre en tiempo $O(n+k)$ y espacio $O(n+k)$ — el arreglo `counts` es $O(k)$, el arreglo de salida es $O(n)$. Cuando k es a lo sumo un múltiplo constante de n, el algoritmo es estrictamente lineal. Cuando k explota (digamos, $k = 2^{32}$ para enteros arbitrarios), el algoritmo colapsa a inutilidad porque el arreglo de conteo solo ya revienta la memoria.

El ordenamiento por base corre en tiempo $O(d(n+b))$ y espacio $O(n+b)$, donde d es el número de dígitos del valor máximo y b es la base. Para tipos enteros de ancho fijo (el caso que encuentras en software real), d es una constante. Así que el tiempo es $O(n)$ y el espacio es $O(n+b)$.

Dicho llanamente: el ordenamiento por conteo es "haz un histograma, luego léelo en orden"; el ordenamiento por base es "haz ordenamiento por conteo en cada dígito, del menos al más significativo, apoyándote en la estabilidad para que las pasadas anteriores sobrevivan". Ninguno de los dos algoritmos realiza una sola comparación entre dos elementos de la entrada. Ambos algoritmos leen la estructura intrínseca de los elementos — los valores mismos, o los dígitos de los valores — y usan esa estructura para calcular las posiciones directamente.

¿Es óptimo?

Sí, dado el modelo, y el argumento es el más barato del libro: la cota inferior para ordenar en cualquier modelo es $\Omega(n)$ — tienes que mirar cada elemento al menos una vez, o un adversario puede cambiar el elemento no examinado para hacer tu salida incorrecta (el mismo argumento del adversario del capítulo 1). El ordenamiento por conteo y el ordenamiento por base alcanzan esa cota exactamente, en el régimen donde sus suposiciones estructurales se cumplen.

Teorema 1. *En cualquier modelo donde el algoritmo deba inspeccionar cada elemento al menos una vez, el ordenamiento requiere $\Omega(n)$. El ordenamiento por conteo y el ordenamiento por base logran $O(n)$ cuando sus suposiciones estructurales se cumplen, y son por tanto estrechamente óptimos en ese modelo.*

Lo que este capítulo cambió fue *qué* cota inferior aplica, no la cota para el ordenamiento en sí. La cota $\Omega(n \log n)$ del modelo de comparaciones del capítulo 4 sigue valiendo para cualquier

algoritmo que solo compare pares de elementos. El ordenamiento por conteo y el ordenamiento por base simplemente no están en ese modelo. Usan una operación más fuerte — indexación de arreglo por valor del elemento, extracción de dígitos — y la cota inferior para ese modelo más rico es la más simple $\Omega(n)$.

Este es el reencuadre con el que el capítulo quiere dejarte: **las cotas inferiores pertenecen a los modelos, no a los problemas**. La cota del árbol de decisión era un hecho sobre lo que las comparaciones pueden resolver. El ordenamiento en tiempo lineal siempre fue posible — los algoritmos de este capítulo solo tuvieron que enriquecer el modelo con operaciones que el mundo de solo comparaciones no permitía.

4. Las cotas inferiores pertenecen a los modelos

El ordenamiento por conteo es correcto porque el arreglo de suma de prefijos dicta colocaciones exactas; el ordenamiento por base es correcto porque cada pasada estable por dígito extiende el orden en un dígito sin perturbar lo que las pasadas anteriores construyeron. Ambos corren en $O(n)$ cuando sus suposiciones estructurales se cumplen — un rango de valores acotado para el ordenamiento por conteo, una representación de dígitos de ancho fijo para el ordenamiento por base. Ambos son óptimos en su modelo — no puedes hacerlo mejor que lineal porque debes leer cada elemento al menos una vez.

Una cota inferior dice lo que un algoritmo *no puede hacer dentro de un modelo dado*, no lo que no puede hacer en absoluto. Cambia el modelo y la cota cambia. La cota $\Omega(n \log n)$ del modelo de comparaciones sobre el ordenamiento es una afirmación sobre lo que las decisiones binarias pueden resolver. El modelo más rico de "mirar dentro del elemento" — direccionamiento directo, extracción de dígitos — obtiene una cota inferior de $\Omega(n)$, porque cada operación extrae más bits de información que una sola comparación. El precio del ordenamiento en tiempo lineal fue relajar la suposición de opacidad escondida en los capítulos 3 a 5.

Ese mismo truco — explotar la estructura intrínseca de la entrada en lugar de tratarla como opaca — vuelve a lo largo del resto del libro. Las tablas hash de la Parte II convierten claves en direcciones, que es exactamente el movimiento "el valor es la dirección" del ordenamiento por conteo generalizado a cualquier tipo de clave. Los árboles de la Parte III siguen usando comparación, pero explotan la estructura de orden de las claves para hacer que cada comparación corte un subárbol a la mitad. Los arreglos de sufijos y los tries de la Parte IV explotan la estructura de caracteres de las cadenas para hacer coincidencia de patrones más rápido de lo que la comparación sola podría lograr jamás. El patrón se repite cada vez que un algoritmo decide dejar de tratar su entrada como opaca y empezar a preguntar qué hay dentro.

En el siguiente capítulo, el foco cambia de los algoritmos teóricos a los que usas. El ordenamiento del mundo real — lo que `sorted()` de Python, `Arrays.sort` de Java, y `std::sort` de C++ hacen bajo el capó — no es ninguno de los algoritmos de esta parte del libro aplicado uniformemente. Es una composición de ellos, cosida a mano para explotar la estructura de las entradas reales. El capítulo es corto, la lección es grande: la optimalidad asintótica es necesaria pero no suficiente.

5. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Sort
{
    public static class CountingSortImpl
    {
```

```
        /// <summary>
        /// Ordena una secuencia de enteros en [0, k] en tiempo O(n + k).
        /// Estable.
        /// </summary>
        public static int[] CountingSort(IReadOnlyList<int> items, int k)
        {
            var counts = new int[k + 1];
            foreach (var x in items)
                counts[x]++;

            for (int i = 1; i <= k; i++)
                counts[i] += counts[i - 1];

            var output = new int[items.Count];
            for (int idx = items.Count - 1; idx >= 0; idx--)
            {
                int x = items[idx];
                counts[x]--;
                output[counts[x]] = x;
            }
            return output;
        }
    }
}
```

Listing 4: `CountingSort.cs` — ordenamiento por conteo

```
using System;
using System.Collections.Generic;

namespace Codex.Sort
{
    public static class RadixSortImpl
    {
        /// <summary>
        /// Ordena enteros no negativos en tiempo O(d * (n + base)).
        /// d = numero de digitos del mayor elemento en la base dada.
        /// LSD (digito menos significativo primero).
        /// </summary>
        public static int[] RadixSort(IReadOnlyList<int> input, int radix = 10)
        {
            if (input.Count == 0) return Array.Empty<int>();
            var items = new int[input.Count];
            for (int i = 0; i < input.Count; i++) items[i] = input[i];

            int maxVal = 0;
            foreach (var x in items)
                if (x > maxVal) maxVal = x;

            long digitPlace = 1;
            while (maxVal / digitPlace > 0)
            {
                items = CountingSortByDigit(items, digitPlace, radix);
                digitPlace *= radix;
            }
            return items;
```

```
        }

        private static int[] CountingSortByDigit(
            int[] items, long digitPlace, int radix)
        {
            var counts = new int[radix];
            foreach (var x in items)
                counts[(int)((x / digitPlace) % radix)]++;

            for (int i = 1; i < radix; i++)
                counts[i] += counts[i - 1];

            var output = new int[items.Length];
            for (int idx = items.Length - 1; idx >= 0; idx--)
            {
                int x = items[idx];
                int d = (int)((x / digitPlace) % radix);
                counts[d]--;
                output[counts[d]] = x;
            }
            return output;
        }
    }
}
```

Listing 5: `RadixSort.cs` — ordenamiento por base (LSD)

6. Notas y lecturas adicionales

El ordenamiento por conteo es folklore; el primer tratamiento publicado se atribuye usualmente a Harold Seward en 1954. CLRS (4.ª ed.) §8.2 da el tratamiento limpio. El ordenamiento por base se remonta a las máquinas tabuladoras de Herman Hollerith en la década de 1880 — fue un algoritmo de hardware mucho antes de serlo de software — y se formalizó para computadoras digitales en la década de 1950. CLRS §8.3 cubre el ordenamiento por base LSD; también hay una variante MSD (dígito más significativo primero) tratada en *Algorithms* (4.ª ed.) de Sedgewick §5.1 que usa recursión de una forma que refleja más al quicksort que al ordenamiento por conteo. El encuadre modelo-vs-problema de la cota inferior de comparaciones — que $\Omega(n \log n)$ es una propiedad del modelo de comparaciones más que del ordenamiento — se trata en CLRS §8.1; es la misma idea sobre la que este capítulo construyó, dicha de dos formas. La relación entre el ordenamiento por conteo y el hashing — ambos siendo "usa el valor como una dirección" — se llama a veces *direccionamiento directo* en la literatura de estructuras de datos; la volverás a encontrar en la Parte II.